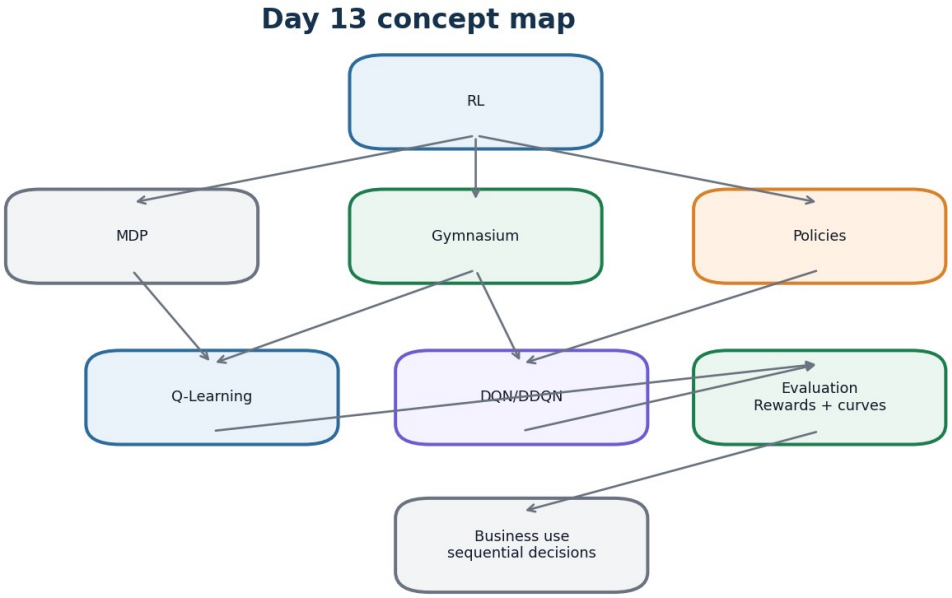
Learning Day 13
Reinforcement Learning Fundamentals

Master Theory Document - Detailed Reworked Version

Ruturaj Mokashi



Complete Day 13 concept map.

Document Purpose and Reading Guide

This document is designed as a detailed learning reference for Reinforcement Learning fundamentals. It explains all MVD topics from Learning Day 13 and also includes supporting concepts from the uploaded lecture material that were not fully implemented in the notebook. The goal is not only to show that the assignment was completed, but also to make the logic understandable later when revising the topic.

The explanation style is intentionally simple. First, each concept is explained with an everyday analogy. Then it is connected to the notebook tasks using CartPole, FrozenLake, Q-Learning, DQN and Double DQN. Finally, the practical interpretation explains what the outputs and charts mean.

How to read this document

Start with the RL loop and MDP sections. Then read CartPole and FrozenLake. After that, read Q-Learning before DQN. DQN makes much more sense once the Q-table idea is clear.

Manual Table of Contents

Coverage Matrix - Nothing Missed

This coverage matrix is split into MVD requirements and additional theory topics. The notebook implemented the MVD tasks. This theory document explains the MVD and expands the missing supporting concepts in detail.

MVD Coverage

MVD level	Requirement	Where explained
Beginner	Create CartPole-v1 and run five random episodes	Sections 7 and 8 explain Gymnasium, CartPole, random policy and reward totals.
Beginner	Explain the four CartPole observation variables	Section 8 explains cart position, cart velocity, pole angle and pole angular velocity.
Beginner	Explain done=True	Section 7 explains terminated, truncated and the practical done condition.
Beginner	Implement angle-based heuristic and compare to random	Section 9 explains why pole angle can be used as a simple rule.
Intermediate	Implement Q-Learning on FrozenLake deterministic	Sections 10 to 13 explain FrozenLake, Q-table, Q update, epsilon-greedy and 10,000 episodes.
Intermediate	Vary alpha and gamma and find fastest convergence	Section 13 explains alpha, gamma and convergence interpretation.
Intermediate	Train on slippery FrozenLake and explain stochastic transitions	Section 14 explains why slippery transitions reduce reliability.
Expert	Implement DQN with 4-64-64-2 network	Sections 17 and 18 explain why the network has this structure.
Expert	Replay buffer 10,000 and target update every 100 steps	Section 19 explains replay memory and target networks.
Expert	Mean reward >195 and smoothed learning curve over 50 episodes	Section 20 explains the threshold and curve reading.
Expert	Implement Double DQN and compare with DQN	Section 21 explains DDQN and overestimation.

Additional Theory Coverage

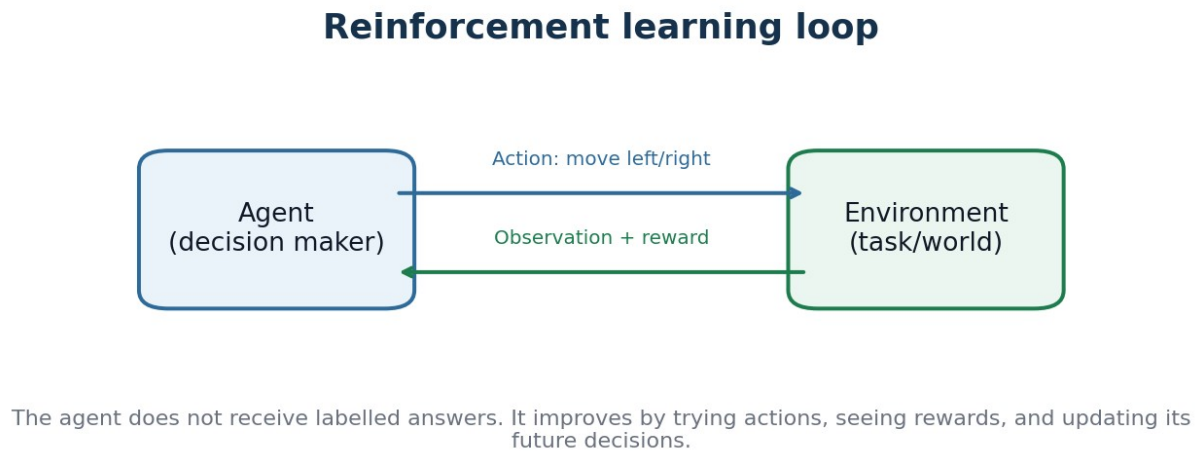
Additional topic	Why included	Where explained
MDPs	Foundation of RL task formulation	Section 3
Policy, value and Q-value	Needed to understand Q-Learning and DQN	Section 5
Bellman equation intuition	Needed to understand bootstrapping	Section 6
Exploration vs exploitation	Needed for epsilon-greedy	Section 12
Monte Carlo and TD learning	Uploaded lecture material and Q-learning context	Section 15
SARSA and on/off-policy learning	Uploaded lecture material and Q-learning comparison	Section 16
Function approximation	Explains why DQN is needed	Section 17
Policy gradients	Mentioned in MVD topic description but not implemented	Section 22
Prioritized replay and dueling DQN	Advanced DQN variants for awareness	Section 22

Business use and limitations	Professional interpretation	Section 23

1. Reinforcement Learning in Simple Language

Reinforcement Learning, or RL, is about learning through trial and error. An agent tries an action, the environment reacts, the agent receives a reward, and the agent slowly learns which actions are better. This is different from supervised learning, where we already have labelled correct answers.

A simple example is teaching a dog a trick. If the dog performs the correct action, it receives a reward. If it performs the wrong action, it receives no reward. Over repeated attempts, the dog learns which behavior gives better outcomes. In RL, the agent is not told the correct action directly. It has to discover useful behavior through experience.



The reinforcement learning loop: action, state feedback and reward.

What we are doing in Day 13

In the Day 13 notebook, we used two Gymnasium environments. CartPole-v1 teaches how an agent interacts with continuous observations. FrozenLake-v1 teaches how a Q-table can learn from a small discrete grid. Later, CartPole is used again for DQN because CartPole has continuous state values and needs a neural network instead of a simple Q-table.

Key idea

RL is useful when the current decision affects future situations. The agent must learn not only what gives reward now, but what leads to better future reward.

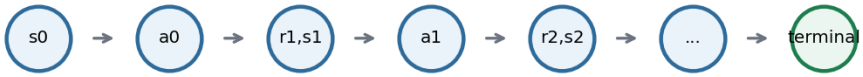
2. Agent, Environment, State, Action and Reward

Every RL problem has a few basic pieces. The agent is the decision maker. The environment is the world or task the agent interacts with. The state or observation is the information the agent sees. The action is the decision the agent takes. The reward is the score returned by the environment after the action.

In CartPole, the agent does not see a picture of the cart. It receives four numbers describing the cart and pole. Based on those numbers, it chooses action 0 or 1. After the action, the environment updates the physics and gives a reward. This repeats until the episode ends.

RL object	Simple meaning	CartPole example	Business example
Agent	Decision maker	The controller choosing left/right	A system choosing next best offer
Environment	World that reacts	CartPole physics simulator	Customers reacting to campaigns
State	Current information	Cart/pole numeric values	Customer status today
Action	Decision	Push left or right	Offer discount or no discount
Reward	Feedback score	+1 for surviving a step	Profit, retention, conversion

One episode is one full attempt



CartPole: episode ends when the pole falls, cart goes out of bounds, or the time limit is reached. FrozenLake: episode ends when the agent reaches goal or hole.

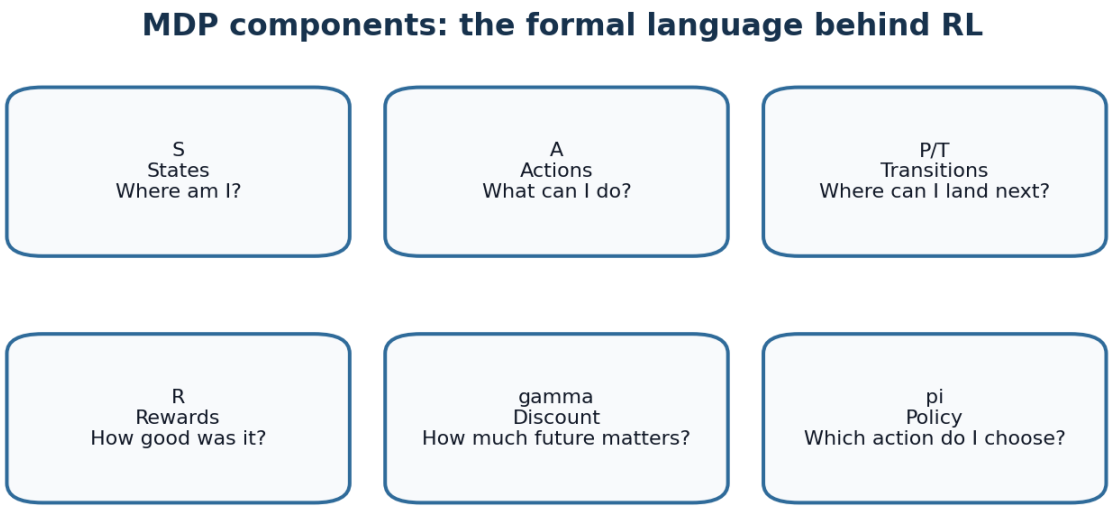
An episode is one full attempt from start to end.

Why episode matters

RL training is measured over episodes because the agent must learn from repeated attempts. One bad episode does not mean the policy is bad. One lucky good episode does not mean the policy is solved. We look at averages and smoothed curves because RL results are naturally noisy.

3. Markov Decision Process

A Markov Decision Process, or MDP, is the formal framework behind many RL problems. It describes states, actions, transitions, rewards and a discount factor. The agent wants to find a policy that chooses actions to maximize expected future reward.



MDP components: states, actions, transitions, rewards, discount and policy.

The Markov property means the current state contains enough information for the next decision. The agent should not need the entire past history if the state is complete. For example, in CartPole, the current position, velocity, angle and angular velocity should be enough to choose the next push direction. If we only had pole angle and did not know velocity, the decision would be less informed.

MDP components with examples

Component	Explanation	Example
State S	All possible situations the agent can be in	Every CartPole observation or every FrozenLake tile
Action A	All actions the agent can choose	Left/right in CartPole, four directions in FrozenLake
Transition P	Probability of next state after action	Slippery FrozenLake may move agent in unintended direction
Reward R	Feedback after transition	+1 for staying alive in CartPole, 1 for reaching FrozenLake goal
Discount gamma	How much future rewards matter	High gamma means the agent values long-term success
Policy pi	Mapping from state to action	Rule, Q-table decision, DQN decision

4. Rewards, Return and Discounting

The immediate reward is the reward received right after one action. The return is the total reward accumulated over time. RL is usually focused on return, not only immediate reward. This is important because many actions that are useful in the long run may not give immediate reward.

Example: In FrozenLake, most steps give reward 0. The agent receives reward only when it reaches the goal. If it only looks at immediate reward, almost every action seems equally useless. Discounted future reward helps the agent connect earlier safe moves with the final goal reward.

Discounted return example

If rewards are R1=1, R2=1, R3=1 and gamma=0.9, the return is 1 + 0.9*1 + 0.9^2*1 = 2.71. Future rewards still matter, but slightly less than immediate rewards.

Gamma	Interpretation	Effect on behavior
0	Future ignored	Agent is extremely short-sighted
0.7	Moderate future value	Good when future is uncertain
0.9	Strong future value	Common in small RL tasks
0.99	Very long-term	Useful for delayed reward, but can make learning slower

5. Policy, Value Function and Q-Value

A policy tells the agent what action to take. It can be random, rule-based or learned. A random policy does not use intelligence. A heuristic policy uses a human rule. A learned policy improves by experience.

A value function estimates how good a state is. A Q-value estimates how good it is to take a specific action in a specific state. In FrozenLake, the agent needs Q-values because it must compare actions from each tile. If moving right has a higher Q-value than moving down, the greedy policy chooses right.

Q-table: rows are states, columns are actions, values estimate future reward.

Plain difference

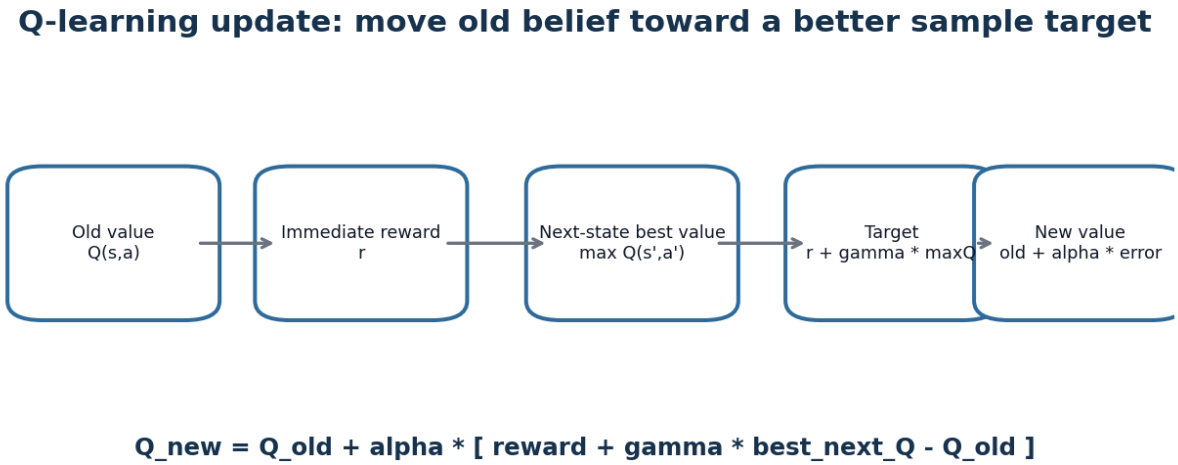
V(s) asks: how good is this state? Q(s,a) asks: how good is this action from this state?

Why Q-values directly support action choice

If we only know V(s), we know whether a state is good or bad, but not directly which action to take. Q(s,a) solves this by giving a score for every action available from the state. The agent can choose the action with the largest Q-value.

6. Bellman Logic and the Q-Learning Update

The Bellman idea is a way to break a long-term decision into smaller pieces. A state-action value should equal the reward now plus the discounted value of what comes next. This is the reason RL can learn from repeated small updates instead of waiting for perfect full information.



Q-learning update: update the old Q-value toward a new target.

The Q-learning update has three important pieces. First, the old Q-value is the agent's previous belief. Second, the sample target combines reward now and the best estimated future value. Third, alpha controls how much we move the old value toward the new target.

Numeric example

Suppose Q_old=0.20, reward=1, gamma=0.90, best_next_Q=0.50 and alpha=0.10. Target = 1 + 0.90*0.50 = 1.45. Difference = 1.45 - 0.20 = 1.25. New Q = 0.20 + 0.10*1.25 = 0.325. The value increased because the experience was better than expected.

7. Gymnasium and the Modern Environment API

Gymnasium is the package used to create RL environments. In Day 13, no CSV files are needed because the environment itself generates experience. The agent calls reset() to start, chooses an action, and calls step(action). The environment returns the next observation, reward, termination status, truncation status and extra information.

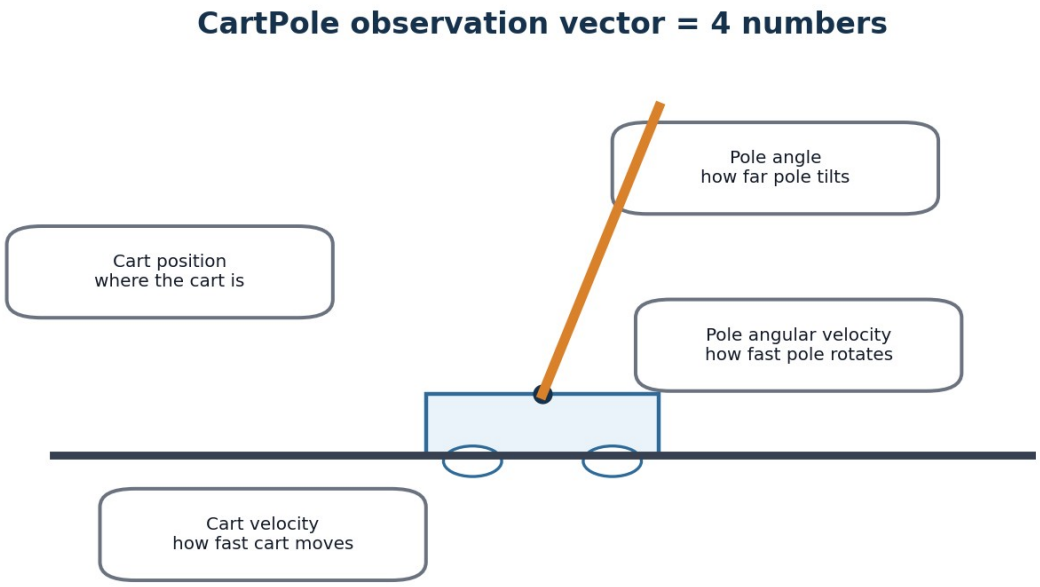
Code concept	Meaning	Why it is needed
env = gym.make(...)	Create environment	Loads CartPole or FrozenLake task
observation, info = env.reset()	Start new episode	Returns initial state
env.action_space.sample()	Random action	Useful for baseline exploration
env.step(action)	Move one step	Returns new observation, reward and end flags
terminated	Task ended naturally	Example: falling into hole or pole falling
truncated	External time limit ended episode	Example: maximum steps reached

done=True in simple language

In old Gym examples, done=True meant the episode ended. In modern Gymnasium, we usually calculate done = terminated or truncated. Terminated means the task itself ended. Truncated means it stopped because of a limit such as maximum episode length.

8. CartPole-v1 in Detail

CartPole is a control task. The cart can move left and right, and the pole must stay balanced. Each step the pole remains upright gives reward. The longer the agent keeps the pole balanced, the higher the episode reward.



CartPole observation vector: four numeric variables used for decision-making.

The observation variables are not just abstract numbers. They have physical meaning. Pole angle is the easiest to understand: if the pole leans right, pushing right can help bring the cart underneath the pole. But a strong policy also uses velocity because the pole may already be rotating quickly. That is why DQN can learn better behavior than the simple angle heuristic.

Observation	Simple interpretation	How a policy might use it
Cart position	Is the cart near the center or edge?	Avoid moving too far out of bounds
Cart velocity	How fast is the cart moving?	Counter fast drift before it becomes dangerous
Pole angle	Is the pole leaning left or right?	Push in the direction that supports the pole
Pole angular velocity	How quickly is the pole falling?	React early before the angle becomes too large

9. Random Policy and Angle Heuristic

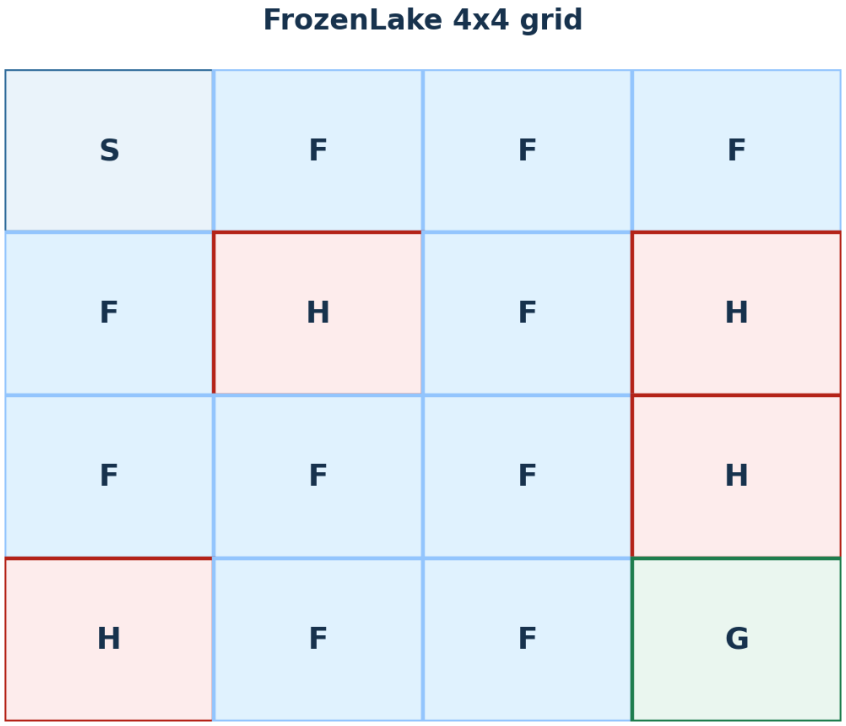
The random policy is intentionally weak. It chooses actions randomly and gives us a baseline. A baseline tells us what performance looks like without learning. This is useful because every trained agent should be compared to something simple.

The angle heuristic is a small rule: if pole angle is greater than 0, push right; otherwise push left. This uses a real physical clue. If the pole leans right, the cart should often move right to catch it. This rule is not perfect, but it is understandable and often better than random.

Policy	Logic	Strength	Limitation
Random	Choose left/right randomly	Good baseline	Does not use observation
Angle heuristic	Use pole angle sign	Simple and explainable	Ignores velocity and position
DQN	Learn from many episodes	Can use all observations	Needs training and can be unstable

10. FrozenLake-v1 in Detail

FrozenLake is a grid-world environment. The agent starts on the start tile and tries to reach the goal. Safe frozen tiles allow movement. Holes end the episode badly. The small number of states makes FrozenLake suitable for tabular Q-Learning.



FrozenLake grid used to explain discrete states and actions.

The agent does not know the best path at the beginning. It must explore. Over many episodes, the Q-table learns which actions from each tile are more likely to eventually reach the goal. This is why the notebook trains for 10,000 episodes: a single episode is not enough experience.

11. Q-Learning Training Workflow

Q-Learning training repeats the same pattern many times. Start an episode, choose an action with epsilon-greedy, step the environment, update one Q-table value, move to the next state, and repeat until the episode ends. Then start another episode.

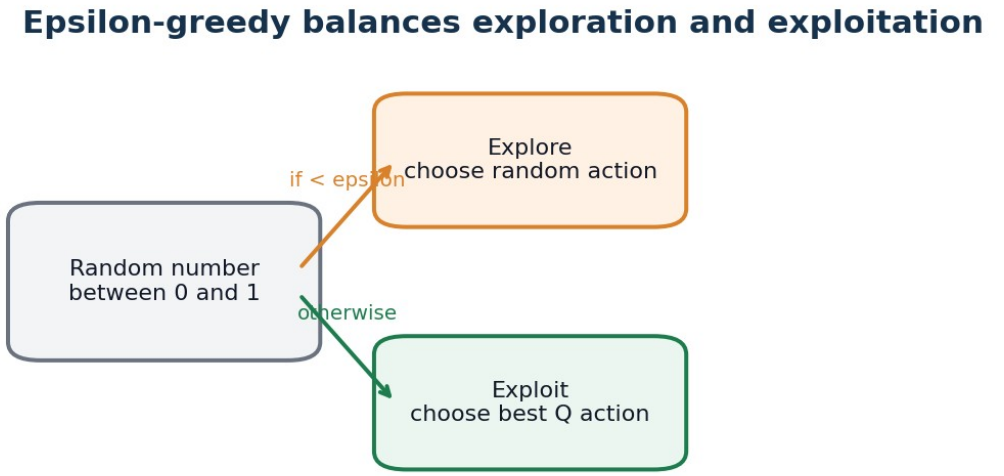
16. Reset the environment and receive the starting state.
17. Choose an action. Sometimes explore randomly, sometimes exploit the best Q-value.
18. Take the action in the environment and observe reward and next state.
19. Calculate the target using reward plus discounted best next Q-value.
20. Update Q(state, action) using alpha.
21. Continue until the episode ends. Repeat for many episodes.

Why last-100 mean reward is useful

Looking at only one episode is noisy. The mean reward over the last 100 episodes shows whether recent behavior is consistently improving.

12. Exploration, Exploitation and Epsilon-Greedy

Exploration is trying actions to gather knowledge. Exploitation is using the best known action. The agent needs exploration because the best action is not known at the beginning. But it also needs exploitation because the goal is to use what it has learned.

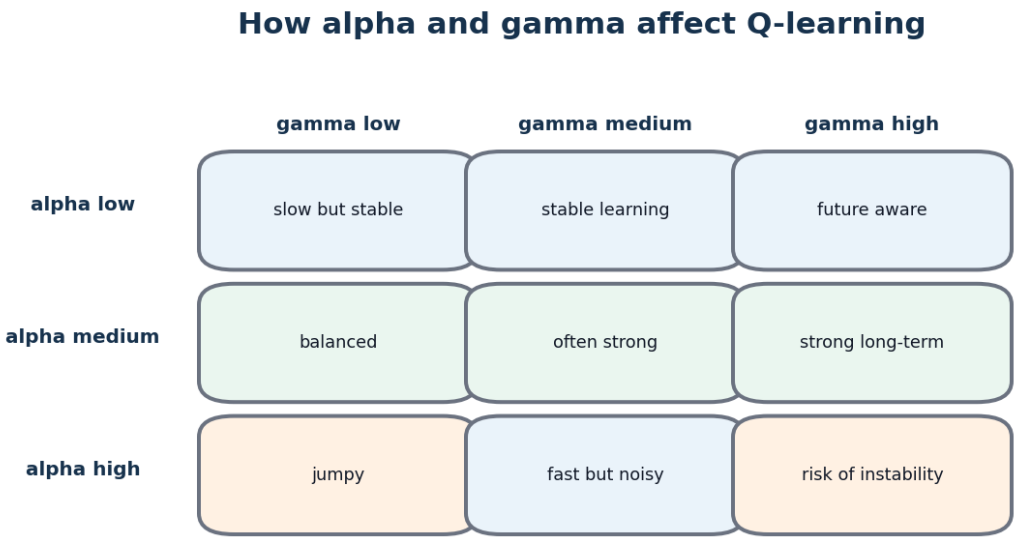


Epsilon-greedy strategy used in the Q-Learning and DQN sections.

If epsilon is high, the agent explores a lot and behaves more randomly. If epsilon is low, the agent mostly follows what it currently believes is best. A decaying epsilon starts with more exploration and gradually becomes more greedy. This is similar to business experimentation: first test many options, then focus on the best-performing option.

13. Alpha, Gamma and Convergence

Alpha and gamma are central Q-Learning parameters. Alpha is the learning rate. It controls how strongly the new experience changes the old Q-value. Gamma is the discount factor. It controls how strongly future rewards are valued.



Alpha and gamma affect convergence speed and stability.

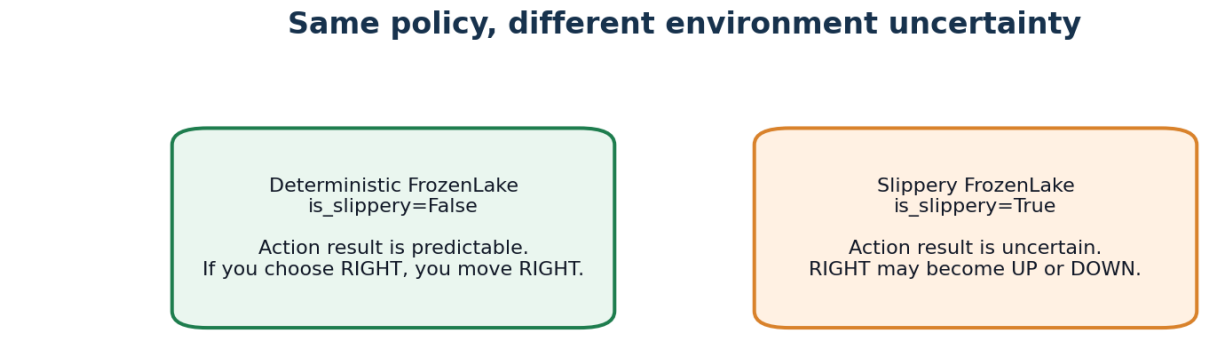
The MVD asks us to test alpha values 0.1, 0.5 and 0.9, and gamma values 0.7, 0.9 and 0.99. The reason is to compare learning behavior. A high alpha can learn quickly but can also jump too much. A high gamma can help long-term planning but may make learning harder when the future is uncertain.

Combination pattern	Likely behavior
Low alpha + low gamma	Slow and short-sighted
Medium alpha + medium gamma	Often balanced and stable
High alpha + high gamma	Can learn fast but may be noisy or unstable
High gamma in slippery lake	May be difficult because future transitions are uncertain

14. Deterministic vs Slippery FrozenLake

In deterministic FrozenLake, an action usually produces the expected movement. If the agent chooses right, it moves right. This makes learning easier because actions have predictable consequences.

In slippery FrozenLake, the action result is stochastic. The agent may intend to move right but slide up or down. This makes learning harder because the same action from the same state can produce different outcomes. The agent must learn what works best on average.



Deterministic and slippery versions of FrozenLake represent predictable and stochastic environments.

Important interpretation

If the slippery version performs worse than the deterministic version, that is expected. It shows that uncertainty in transitions makes control harder.

15. Monte Carlo, TD Learning, SARSA and Learning Types

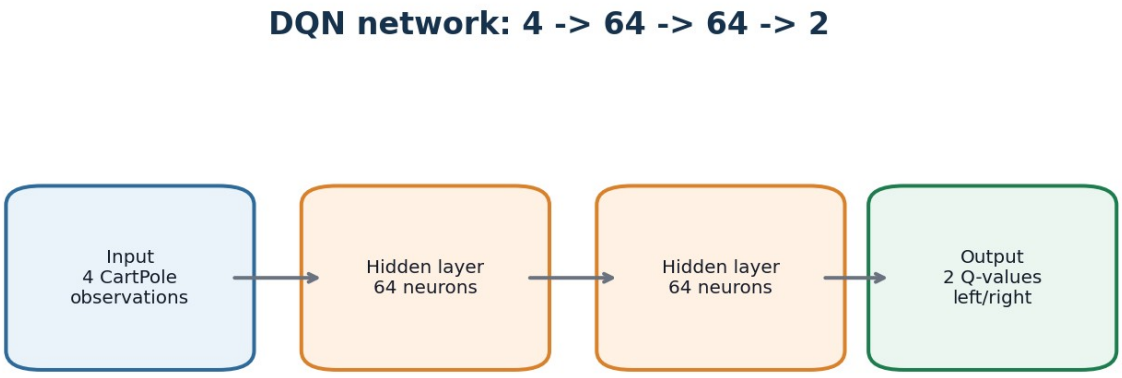
The uploaded lecture material includes several RL learning approaches. They are important for understanding where Q-Learning fits. Monte Carlo learns from complete episodes. TD learning learns from individual steps. Q-Learning is a TD control method because it updates Q-values after each transition.

Approach	Learns from	Simple explanation
Monte Carlo	Full episode	Wait until the end and learn from total return
TD learning	One transition	Learn immediately using reward plus next-state estimate
SARSA	Current policy action	On-policy control method
Q-Learning	Best next action estimate	Off-policy control method
Model-based RL	Estimated transition/reward model	Learn the environment model first, then plan
Model-free RL	Values or policy directly	Learn what to do without building full model

On-policy means the agent learns about the same policy it is using to behave. SARSA is on-policy because it updates using the action actually selected by the current policy. Off-policy means the agent can behave exploratorily but learn about a different target policy. Q-Learning is off-policy because it uses the best next action in the update.

16. From Q-Table to DQN

Q-tables are practical for small discrete environments. They are not practical for large or continuous state spaces. CartPole observations are continuous numbers, so there are too many possible states to store in a table. DQN solves this by replacing the Q-table with a neural network.



DQN architecture required by the MVD: 4 inputs, two hidden layers of 64, and 2 outputs.

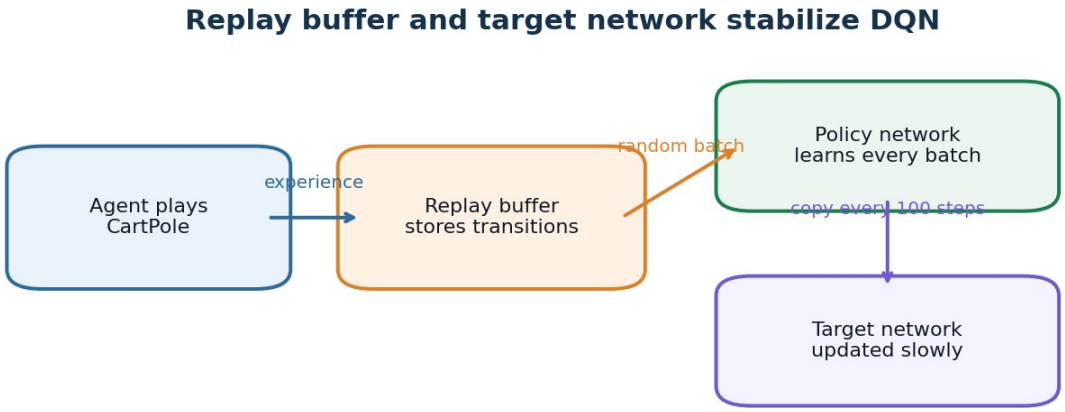
The DQN network receives a state and outputs one Q-value per action. In CartPole, the input has 4 variables and the output has 2 values. The agent chooses the action with the highest predicted Q-value, except during exploration.

What the network output means

If the output is [32.5, 41.0], the model currently believes action 1 has higher expected future reward than action 0. The greedy action is therefore 1.

17. Experience Replay and Target Network

DQN training can be unstable because the data is sequential and the target changes over time. Experience replay and target networks are used to reduce this instability.



Replay buffer and target network are two stabilizing mechanisms in DQN.

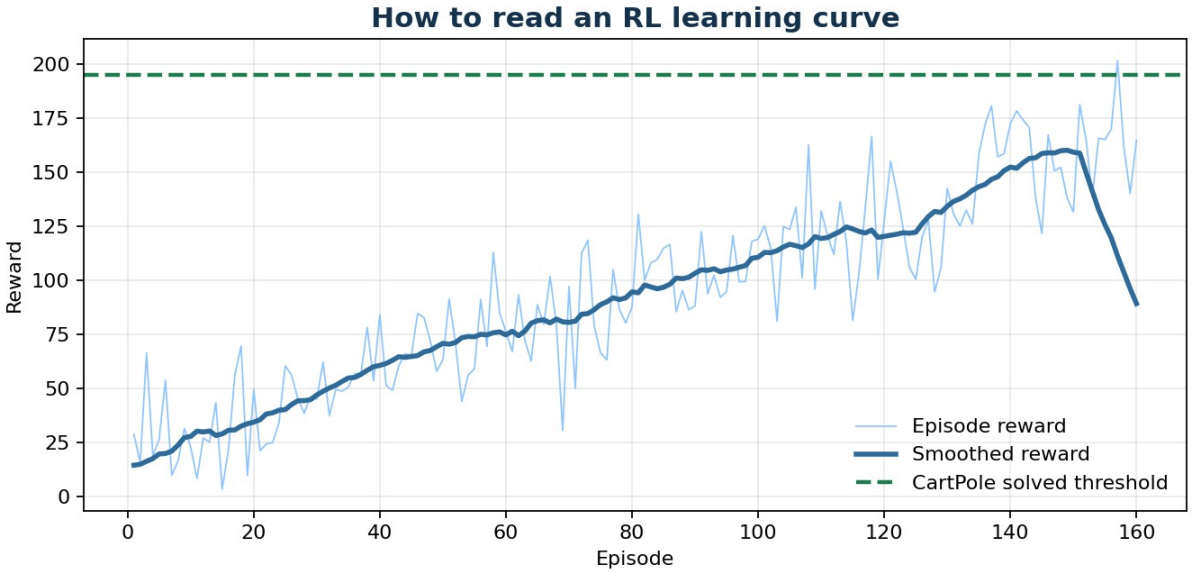
The replay buffer stores transitions such as state, action, reward, next state and done. During learning, the model samples a random mini-batch from this memory instead of only learning from the newest transition. This breaks strong correlation between consecutive samples.

The target network is a delayed copy of the policy network. It is used to calculate the target Q-values. In the MVD notebook, the target network is updated every 100 steps. This makes the learning target more stable.

Component	Without it	With it
Replay buffer	Training samples are highly correlated	Batches are mixed from past experience
Target network	Targets move too quickly	Targets change more slowly
Epsilon decay	Agent may stay too random	Exploration decreases over time

18. DQN Learning Curves and the >195 Threshold

The MVD asks when the agent achieves a mean reward greater than 195. This is a performance threshold for CartPole. The important word is mean. A single reward above 195 is not enough. We need a moving average or stable mean reward because one lucky episode does not prove the agent has learned reliably.

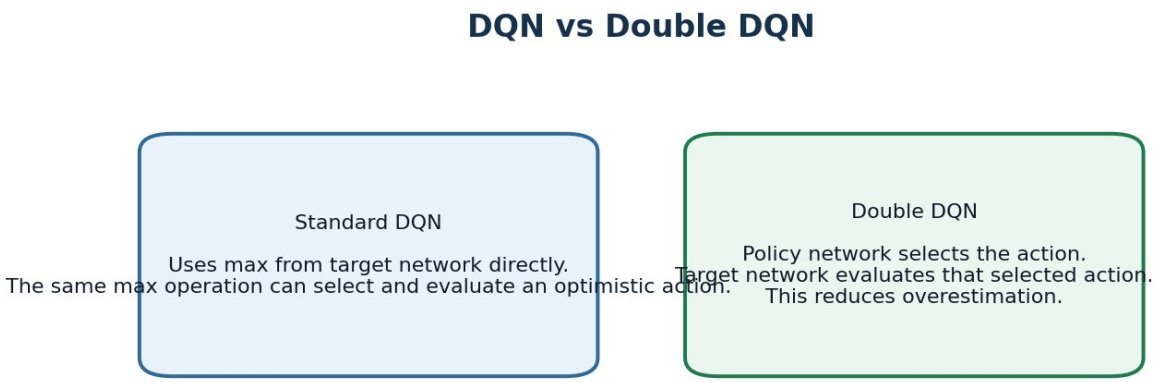


A smoothed reward curve is easier to interpret than raw noisy episode rewards.

If the smoothed curve rises over time, the agent is learning. If it stays flat, learning is not improving. If it rises and then falls, learning may be unstable. DQN and DDQN curves can differ because DDQN reduces overestimation bias, but due to randomness, one run is not always enough for a final scientific conclusion.

19. Double DQN

Standard DQN uses a max operation over estimated Q-values. When estimates contain noise, the max operation tends to select overly optimistic values. This is called overestimation bias. Double DQN reduces this problem by separating action selection and action evaluation.

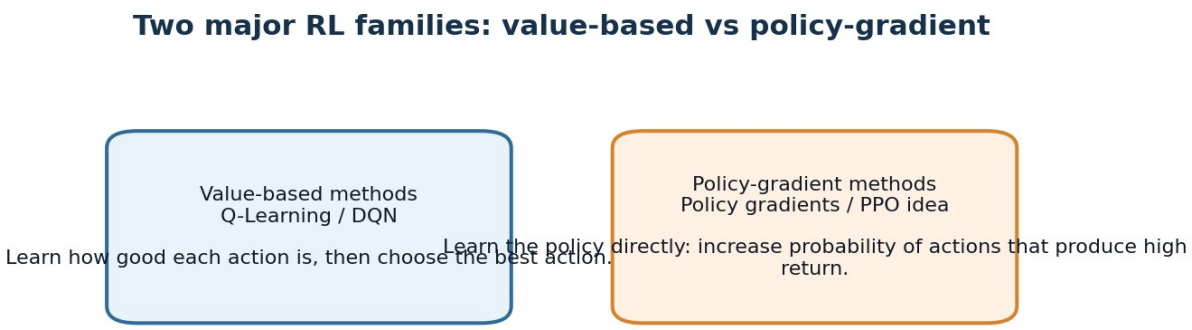


Double DQN separates action selection and evaluation.

In Double DQN, the policy network selects the best next action. The target network then evaluates the value of that selected action. This small change can make learning more stable because one network does not both choose and evaluate the same optimistic estimate.

20. Policy Gradients and Advanced Topics Not Implemented

The MVD topic description mentions policy gradients, but the practical tasks focus on Q-Learning, DQN and DDQN. Policy gradients are a different family of RL algorithms. Instead of learning Q-values first and choosing the best action, policy-gradient methods learn the policy directly.



Value-based methods versus policy-gradient methods.

Policy-gradient methods increase the probability of actions that lead to high return and decrease the probability of actions that lead to low return. This is important in modern RL, especially when action spaces are continuous or when we want stochastic policies.

Other advanced topics for awareness

- Prioritized replay: sample important or surprising transitions more often instead of sampling uniformly.
- Dueling DQN: separately estimates state value and action advantage inside the network.
- Policy optimization methods: learn action probabilities directly, often used in advanced continuous-control tasks.
- Offline RL: learns from fixed historical data instead of live environment interaction.
- Safe RL: adds constraints so the agent does not explore dangerous or unacceptable actions.

21. Business Interpretation and Practical Limits

RL is useful when a decision today changes the next situation and affects future reward. It is not the correct tool for every problem. Many business problems are better solved with regression, classification, forecasting or optimization. RL becomes relevant when the problem is sequential and action-based.

Business use case	State	Action	Reward
Next-best offer	Customer behavior and profile	Offer A, offer B, no offer	Retention value minus discount cost
Inventory control	Stock, demand, supplier lead time	Order quantity	Service level minus stockout and holding cost
Dynamic pricing	Demand, inventory, competitor signals	Price choice	Margin and conversion
Operations scheduling	Queue, deadlines, resources	Assign next job	Delay reduction and throughput
Recommendation system	User history and context	Recommend item	Engagement or conversion over time

Practical warning

RL can be risky when exploration affects customers, money, safety or compliance. Real deployments often need simulation, guardrails, offline evaluation and controlled experiments.

22. Notebook Workflow Explained

The Day 13 notebook was built in three practical blocks: Beginner, Intermediate and Expert. Each block corresponds directly to the MVD requirements.

Notebook block	What was done	What it proves
CartPole random policy	Ran five random episodes	Shows basic environment interaction and baseline reward
CartPole heuristic	Used pole angle rule	Shows how a simple state-based rule improves over random
FrozenLake Q-Learning	Trained a Q-table	Shows model-free tabular control
Alpha/gamma experiments	Tested 9 combinations	Shows parameter impact on convergence
Slippery FrozenLake	Trained under stochastic transitions	Shows uncertainty makes RL harder
DQN	Used neural network Q-function	Shows approximate Q-learning for continuous states
DDQN	Compared to DQN	Shows overestimation-reduction idea

23. How to Read the Notebook Charts

The notebook charts are not decoration. Each chart answers a learning question. Reward charts show whether the agent improves. Moving average charts smooth noisy RL performance. Comparison charts show whether one policy or parameter setting performs better than another.

Chart type	Question it answers	How to read it
Episode reward bar chart	How did each episode perform?	Higher bars mean longer survival or better return.
Mean reward comparison	Did heuristic beat random?	Compare average reward by policy.
Q-learning moving average	Is learning improving over time?	Rising curve means better recent performance.
Alpha/gamma heatmap	Which combination converged fastest?	Look for higher reward and earlier stability.
DQN/DDQN curve	Which deep RL agent learned better?	Compare smoothed curves and final mean reward.
Threshold marker	Did CartPole reach >195 mean reward?	Marker shows the first stable success point, if reached.

24. Final Recap

Day 13 introduces reinforcement learning as learning from interaction. The agent does not learn from labelled rows like supervised learning. It learns by trying actions, receiving rewards and improving its policy. CartPole shows continuous control, FrozenLake shows tabular Q-Learning, and DQN shows how neural networks can approximate Q-values when tables are not practical.

Final memory anchor

Random policy gives a baseline. Heuristic policy uses a simple rule. Q-Learning learns a Q-table. DQN learns Q-values with a neural network. Double DQN improves DQN by reducing overoptimistic Q-value estimates.

25. Glossary

Term	Meaning
Agent	Decision maker that chooses actions.
Environment	Task or world that reacts to actions.
State / observation	Information the agent receives at a time step.
Action	Choice made by the agent.
Reward	Feedback score after an action.
Episode	One full run from reset to end.
Policy	Rule or model that maps states to actions.
Return	Total discounted future reward.
Q-value	Expected future reward for taking an action in a state.
Q-table	Table of state-action values.
Epsilon-greedy	Explore randomly sometimes, otherwise choose best known action.
Alpha	Learning rate for Q-value updates.
Gamma	Discount factor for future rewards.
DQN	Deep Q-Network using a neural network for Q-values.
Replay buffer	Memory of past transitions.
Target network	Slowly updated network used for stable DQN targets.
DDQN	Double DQN, which reduces overestimation bias.
Policy gradient	RL method that learns action probabilities directly.

26. Final Coverage Checklist

Topic	Status
CartPole-v1 random policy	Covered
CartPole observation variables	Covered
done=True / terminated / truncated	Covered
Rule-based heuristic	Covered
FrozenLake deterministic Q-Learning	Covered
Epsilon-greedy policy	Covered
10,000 training episodes	Covered
Mean reward last 100 episodes	Covered
Alpha/gamma experiment	Covered
Fastest convergence explanation	Covered
Slippery FrozenLake	Covered
Stochastic transitions explanation	Covered
DQN architecture 4-64-64-2	Covered
Replay buffer 10,000	Covered
Target network update every 100 steps	Covered
Mean reward >195 tracking	Covered
50-episode smoothed learning curve	Covered
Double DQN comparison	Covered
MDP theory	Covered
Policy and value function theory	Covered
Q-values and Bellman update	Covered
Monte Carlo and TD learning	Covered
SARSA/on-policy/off-policy	Covered
Function approximation	Covered
Policy gradient overview	Covered
Business interpretation	Covered
Glossary	Covered

References and Source Base

This document uses the uploaded Day 13 lecture material and additional official or widely accepted RL sources for clear terminology and implementation context.

Reference	Used for
Uploaded RL lecture PDFs	MDPs, value functions, Q-states, Q-values, model-free learning, TD learning, Q-Learning, exploration, function approximation.
Gymnasium documentation	CartPole-v1, FrozenLake-v1, reset/step API, terminated and truncated meanings.
PyTorch DQN tutorial	DQN implementation structure, replay memory and target network style.
Sutton and Barto, Reinforcement Learning: An Introduction	Core RL theory, returns, value functions, TD learning and control methods.
Mnih et al., Human-level control through deep reinforcement learning	DQN, experience replay and target network context.
van Hasselt et al., Deep Reinforcement Learning with Double Q-learning	Double DQN and overestimation bias.
OpenAI Spinning Up	Policy gradient intuition and broader RL terminology.